



Artificial Intelligence (CIS2031-N)

Lab introduction on Program Flow Control

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk/computing

The `print()` function

- The `print()` function will write some text to the screen.
- The `print()` function can take multiple arguments separated by a comma.
- The comma introduces a space in the output separating the printed content of adjacent arguments. The argument `end=""` can be used in the print function to stop a linefeed prior to next use of a `print()` statement (we'll use this in a later session).

```
print()  
print("Result is: ", result)  
print(x, "*", y, "=", (x * y))
```

The `input()` function

- The `input()` function will get a value from the user.
- The `input()` function should normally be given with a prompt to observe good user interface practice (single argument only)
- The prompt can be a literal (as indicated) or a string variable.
- The `input()` function would normally include a variable name assignment to record whatever was entered in response.

```
input()  
input("Please enter something: ")  
name = input("And your name is?")
```

The input statement returns a character string type!

Casting

- Casting is the process of converting a value of one type to another.
- For example, if we read two numbers from the user as strings, we cannot add them together until we turn them into integers! To cast to values of a different type, we use functions including:
 - `float(x)` - converts x to a float
 - `int(x)` - converts x to an integer
 - `str(x)` - converts x to a string

```
x = input("Enter x: ")
y = input("Enter y: ")

print(x * y) # This won't work!

print(int(x) * int(y)) # This will!
# Notice the use of casting here!
```

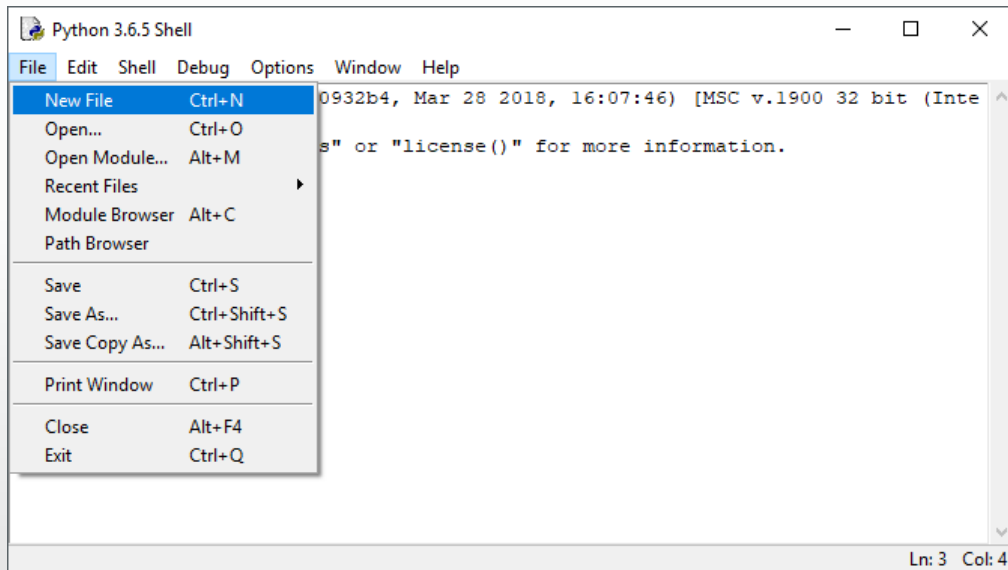
Operators

- Operators are functions that work on *operands*.
- Examples, with operands **x** as **8** and **y** as **3** are shown on the right.
- These are called *binary* operators. *Binary* just means '2', and because these operators take 2 operands, that's how they get their name.
- This has nothing to do with the concept of binary numbers (i.e. 1s and 0s)!

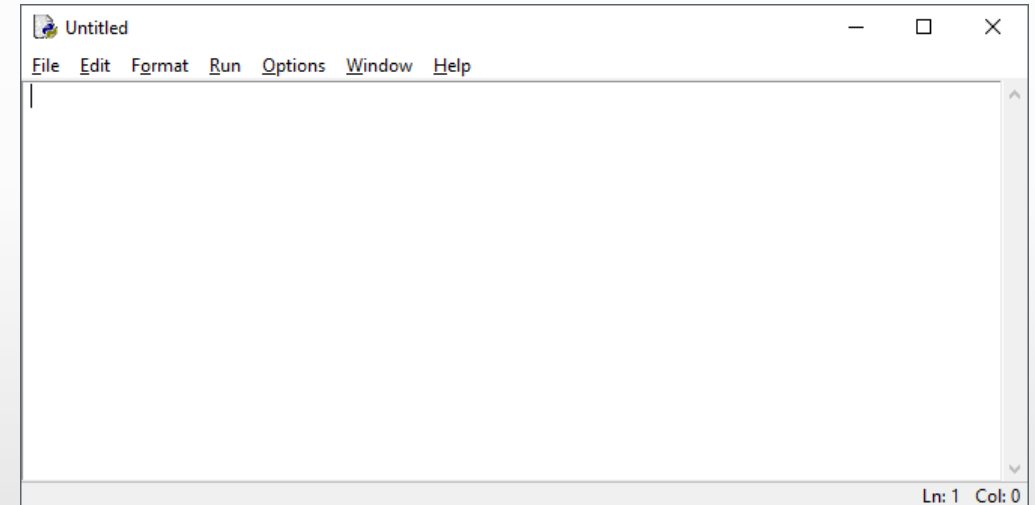
Operation	Python	Example
Addition, subtraction	+ or -	$x + y = 11$
Multiplication	*	$x * y = 24$
Division	/	$x / y = 2.67$
Exponentiation (Power)	**	$x ** y = 256$
Integer division	//	$x // y = 2$

Creating Python modules from IDLE

Create a new file in IDLE...

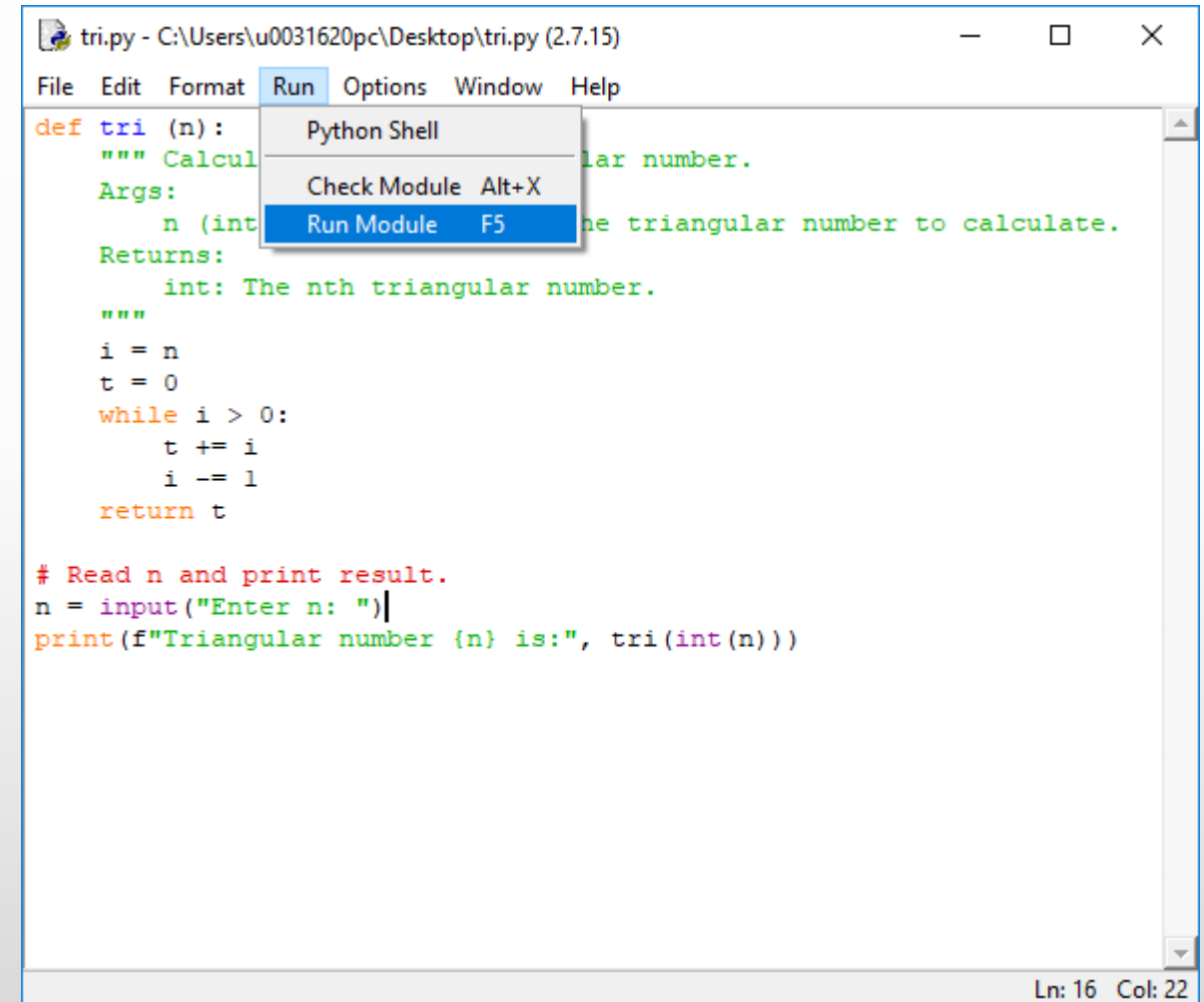


Write it and run with F5!



Creating Python Modules from IDLE (cont.)

- Python files must be saved with the .py extension!
- You can click “Run” in the IDE to run your code, or you can use the F5 shortcut key!



```
tri.py - C:\Users\u0031620pc\Desktop\tri.py (2.7.15)
File Edit Format Run Options Window Help
def tri (n):
    """ Calcul... lar number.
    Args:
        n (int... the triangular number to calculate.
    Returns:
        int: The nth triangular number.
    """
    i = n
    t = 0
    while i > 0:
        t += i
        i -= 1
    return t

# Read n and print result.
n = input("Enter n: ")
print(f"Triangular number {n} is:", tri(int(n)))

Ln: 16 Col: 22
```

Okay, recap over!

On to this week's material...

Learning objectives

Conditionals

- Introduce Boolean logic and Boolean algebra. What is it and how can it help us when we're programming?
- Understand what we mean by *branching* when it comes to selecting which code statements to execute.
- Understand how to use the three main conditional keywords in Python: `if`, `elif` and `else`.

Iteration

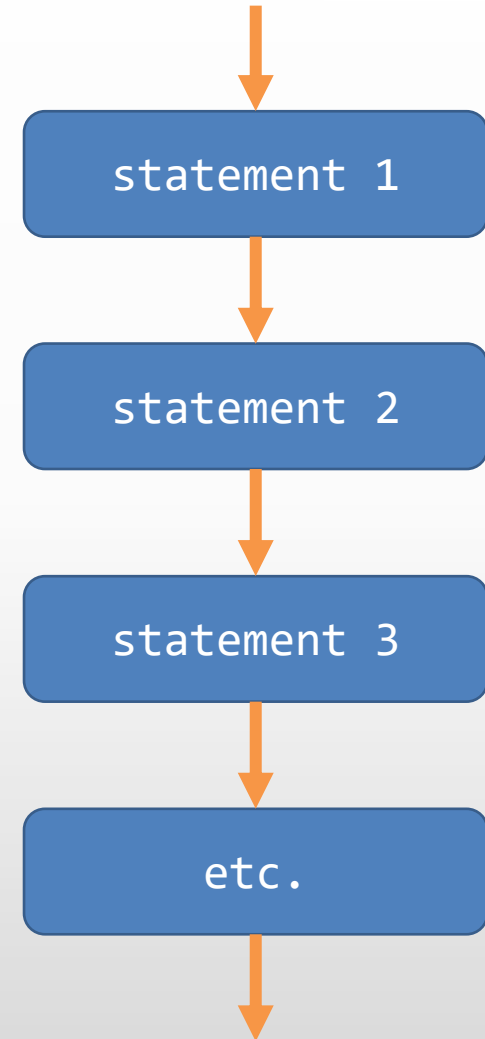
- Explore what iteration is and how we can use it to control the flow of a program.
- Understand what we mean by a *list* in Python.
- Understand the sort of problems that we use lists to solve, and some basic operations we can perform on them.
- Explore how iteration and lists can work together.

Flow control

What is it?

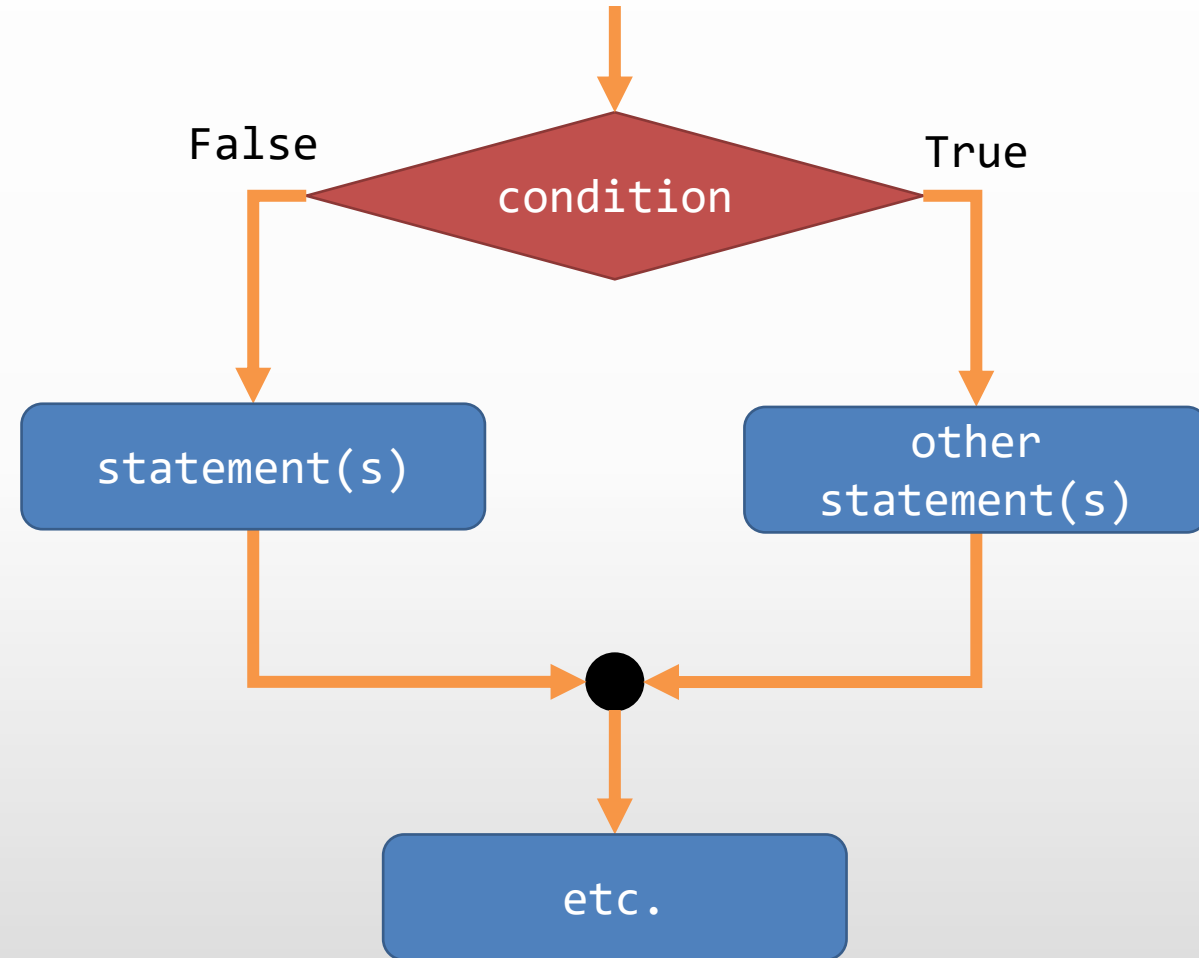
Quick recap: Sequence

- Normally, in Python, commands are executed in a *sequence*.
- That is to say, one after the other in the order they're written.
- Every program we've written together so far (unless you've read ahead!) has been sequential.
- What if, however, we want to execute code only in certain cases?



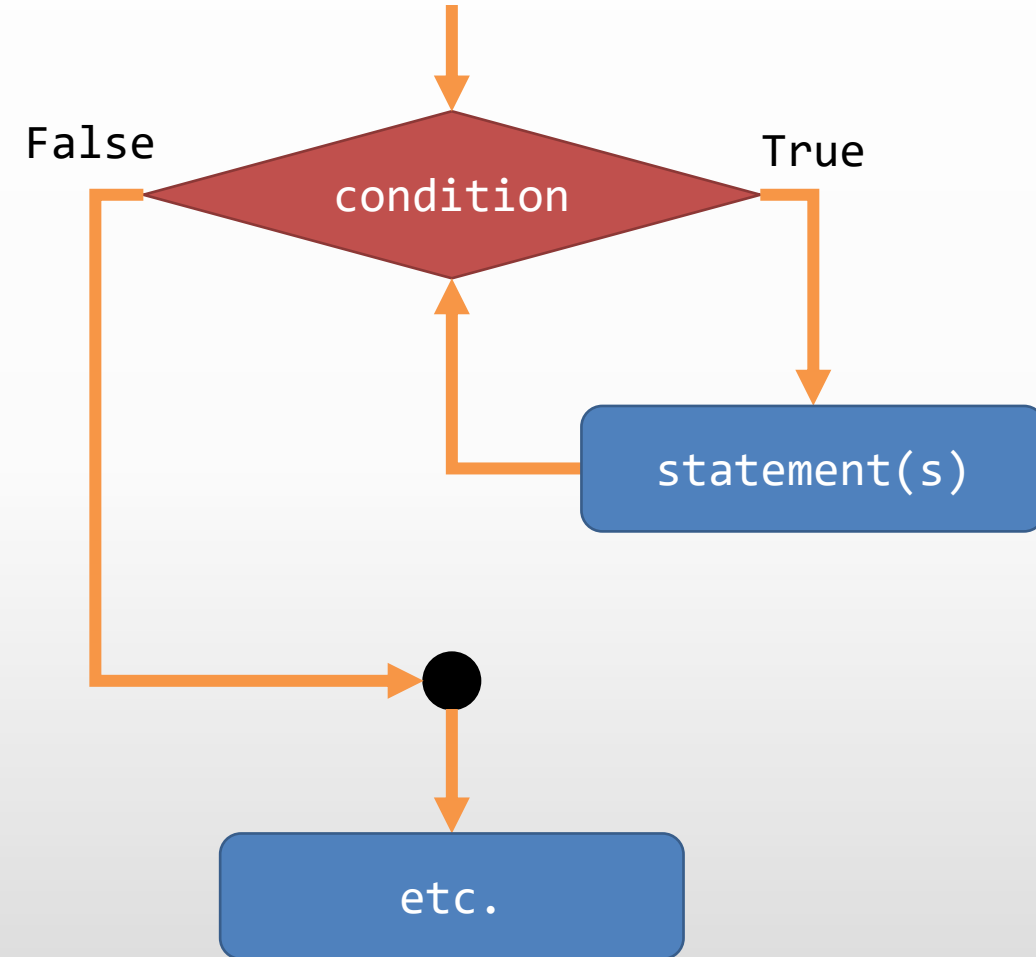
Selection (conditionals/if)

- If we only want to execute code when certain conditions are met, we need to use a *conditional statement*.
- In Python, these are also called *if statements*.
- As you can imagine, these are *extremely* useful. Without conditional statements, a program would do the same thing every time!



Iteration (looping)

- Iteration is what really makes programming useful.
- It is also called *looping*.
- For example, rather than writing the same code a million different times to send a million e-mails (resulting in a very large program!) you can write the e-mail sending code once, then loop through a list of e-mail addresses.

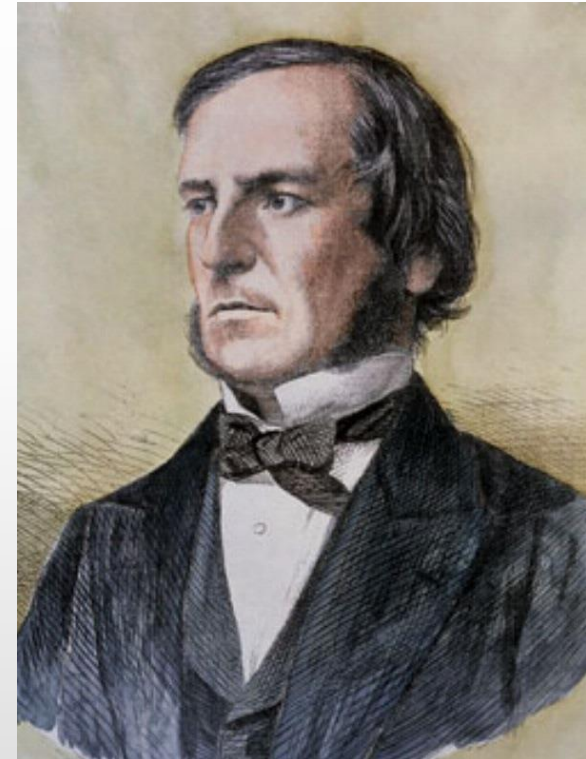


Boolean logic

est. 1854...

Introducing... Boolean logic!

- Boolean logic is named after the mathematician George Boole, who wrote a book containing his ideas in 1854 called *The Laws of Thought*.
- The core idea is that the simplest sort of value you can get is the answer to a yes or no question.
- In Boolean logic, the only values you can get are **true** and **false**. There is no in-between and there is no equivalent of *null*.
- Which of the following are Boolean questions?
 - How many apples are in the basket?
 - Are there apples in the basket?
 - Are there 5 apples in the basket?
 - Are there more apples in the basket than in the bucket?



Boolean logic (part 1.)

- Boolean logic sounds scary, but it's the simplest logic you can imagine!
- We don't deal with numbers; we deal with only two values: **true** and **false**.
- Instead of addition, subtraction, etc. we use the Boolean operators **and**, **or**, and **not**.
- Let's try the code on the right in the REPL. What's its semantics?

```
>> True
>> False
>> True and False
>> True or False
>> (True or False) and True
```


Boolean logic (part 2.)

- We have a few more technical terms for ‘and’, ‘or’ and ‘not’ in Boolean logic.
- ‘And’ is technically called ‘conjunction’.
- ‘Or’ is technically called ‘disjunction’.
- ‘Not’ is technically called ‘negation’.
- We also use some special symbols when we write Boolean *algebra*, but **not** when we write Python (see right):
 - $x \wedge y$ means ‘x and y’
 - $x \vee y$ means ‘x or y’
 - $\neg x$ means ‘not x’
 - T means ‘true’ and $\perp$ means ‘false’

Name	Symbol	Python	Example
Conjunction	$\wedge$	and	<pre>>> True and True True >> True and False False</pre>
Disjunction	$\vee$	or	<pre>>> True or True True >> True or False True</pre>
Negation	$\neg$	not	<pre>>> not True False >> not False True</pre>

Conditional operators (checks)

Name	Python	Example	Result
Equality (value)	<code>==</code>	<code>(3 + 3) == 6</code>	True
Inequality (value)	<code>!=</code>	<code>(3 + 3) != 6</code>	False
Greater than or equal to	<code>>=</code>	<code>4 >= 4</code>	True
Less than or equal to	<code><=</code>	<code>3 <= 2</code>	False
Greater than	<code>></code>	<code>3 > 4</code>	False
Less than	<code><</code>	<code>6 < 12</code>	True
Equality (address)	<code>is</code>	<code>float(3) is float(3)</code>	False
Inequality (address)	<code>is not</code>	<code>float(3) is not float(3)</code>	True
Membership	<code>in</code>	<code>3 in [1, 2, 3]</code>	True
Non-membership	<code>not in</code>	<code>3 not in [1, 2, 3]</code>	False

Truth tables

- Truth tables are very useful for modelling how our programs are going to run.
- By simulating different conditions, we can map out exactly how our program can run.
- We can write truth tables by hand, but certain software can generate and automatically analyse truth tables with millions of rows!
- Can you see how the truth table on the right works?

<i>a</i>	<i>b</i>	<i>a</i> = 42	<i>b</i> < 100	<i>a</i> ∧ <i>b</i>
42	99	⊤ (true)	⊤	⊤
42	110	⊤ (true)	⊥	⊥
3	99	⊥ (false)	⊤	⊥
3	110	⊥ (false)	⊥	⊥

Flow control in Python code

The If statement.

The `if` statement (part 1.)

- To the right, there's a Python program that gets two integers from the user and then tells the user which is biggest.
- First, with the `if` statement, we're asking "If x is greater than y, print this."
- Then, with the `elif` statement, we're saying "otherwise, if y is greater than x, print this."
- Finally, the `else` statement says, "If none of the above (i.e. *otherwise*), print this."

```
x = int(input("Enter x: "))
y = int(input("Enter y: "))

if x > y:
    print(x, "is biggest!")
elif y > x:
    print(y, "is biggest!")
else:
    print("Both are equal!")
```

The `if` statement (part 2.)

- We can join conditions together into so-called compound conditions, using `and/or` and `not`.
- The small program on the right checks if the number given is both greater than 100, and even!
- Please experiment with this, it's the best way to learn!

```
x = int(input("Enter a big even int:"))

if x > 100 and x % 2 == 0:
    print("Cool number!")
else:
    print("Very uncool number...")
```

Nested conditionals

- We can put if statements inside if statements! This is called *nesting*.
- This can be very useful in many cases, particularly when you want your program to enter a whole different branch of behavior in certain cases.
- In the example on the right, we ask the user their age, and if they're under 18 we ask them if they have an adult with them. Otherwise we kick them out!

```
age = int(input("Enter your age:"))

if age < 18:
    adult = input("Adult with you?")
    if adult != "yes":
        print("Access denied!")
        exit(0)
else:
    print("You can proceed.")
```

Looping in Python

Also known as *iteration*.

The `while` statement (loop)

- To the right, there's a Python program that will run forever until the user types "yes".
- The `while` statement (or *while loop*) works in a surprisingly similar way to the `if` statement, though they're used for very different purposes.
- The `while` statement will run around and around in a loop while its condition (or *predicate*) is true. When it becomes false, the loop stops.

```
q = input("Wanna quit?")

while q != "yes":
    q = input("How about now?")
```

Making a list (part 1.)

- Rather than just putting a single value in a variable, we can put a whole list of values in there instead.
- Then, using a **while** loop and *list indexing*, we can print every number individually one at a time.
- If we want to access the first number in the list called `nums` on the right, for example, we'd write `nums[0]`. So `nums[0] == 2` and `nums[1] == 4`.

```
nums = [2, 4, 6, 8]
current = 0

while current < len(nums):
    print(nums[current])
    current += 1
```

Making a list (part 2.)

- So, look at what we're doing here. First, we create a list of 4 numbers called `nums`.
- Then we set a counter called `current`.
- Then, `while` `current` is less than the length (`len`) of `nums`, we print the *member* of `nums` at the *index* specified by the counter.
- When we say *index*, we mean the number's position in the list, with the first item's index being 0.
- Then we add 1 to counter and loop around again.

```
nums = [2, 4, 6, 8]
# We say that nums has 4
# members at indices 0 through
# to 3.
current = 0
while current < len(nums):
    print(nums[current])
    current += 1
```

The `for` statement: Cleaner iteration!

- The `for` statement is a different type of loop, which allows you to iterate much more cleanly over lists.
- This loop structure will go through your list one at a time and put each member in a variable for you.
- When there are no more list items left, the loop will quit.
- The program on the right behaves identically to the one on the last slide!

```
nums = [2, 4, 6, 8]
```

```
for num in nums:  
    print(num)
```

Any questions?

Module Leader Dr Alessandro Di Stefano

Email A.DiStefano@tees.ac.uk

Office G0.44A (Greig building)

See Blackboard for online materials: <https://eat.tees.ac.uk>